

Informe de Validación y Optimización de Base de Datos

Joanthan Jesús Flores Reyes

11 de julio de 2026

1. Resultados de Consultas con Datos de Prueba

Se poblaron las tablas con un lote de prueba (30 productos, 15 clientes, 20 pedidos) para ejecutar las vistas analíticas. A continuación, se presentan los resultados obtenidos:

Consulta 1: Productos disponibles por categoría (Muestra)

Categoría	Producto	Precio (\$)	Stock
Almacenamiento	Gabinete Externo Orico para SSD	450.00	35
Audio	Audífonos Inalámbricos Xiaomi	850.00	50
Componentes PC	Pasta Térmica de Alto Rendimiento	200.00	100
Monitores	Monitor de Oficina 24 pulgadas FHD	2200.00	15

Consulta 2: Clientes con pedidos pendientes y gasto histórico

Nombre	Correo	Pedidos Pendientes	Total Gastado (\$)
Jorge Salinas	jorge.s@email.com	2	9300.00
Daniela Torres	dany.t@email.com	1	900.00
Fernando Silva	fer.silva@email.com	1	8900.00

Consulta 3: Top productos con mejor calificación promedio

Producto	Promedio Calificación	Total Reseñas
Tarjeta Gráfica AMD Radeon RX 7800 XT	5.00	1
Monitor Gaming 144Hz 27 pulgadas	5.00	1
Audífonos de Estudio Cerrados	5.00	1

2. Índices Creados y su Impacto (Análisis EXPLAIN)

Para optimizar el rendimiento del sistema, se crearon los siguientes índices no agrupados: `idx_productos_categoria`, `idx_pedidos_cliente` y `idx_productos_nombre`.

Al utilizar el comando `EXPLAIN` en consultas que filtran por categoría o cliente, se observó el siguiente impacto:

- **Antes de los índices:** El plan de ejecución mostraba un tipo de búsqueda *ALL* (Full Table Scan), obligando al motor a recorrer el 100 % de las filas.
- **Después de los índices:** El tipo de búsqueda cambió a *ref*. La métrica *rows* se redujo drásticamente, ya que el motor accede directamente a los registros coincidentes mediante la estructura del árbol de índices, reduciendo los tiempos de respuesta y el consumo de CPU.

3. Pruebas de Procedimientos Almacenados

Se validó la robustez de las reglas de negocio ejecutando pruebas funcionales bajo diferentes escenarios.

Escenario 1: Intento de reseña sin compra previa (sp_registrar_reseña)

Se simuló a un usuario intentando calificar un artículo que no ha adquirido. El procedimiento bloqueó la acción:

```
Error Code: 45000. Error: Solo los clientes que han comprado el producto pueden
dejar una reseña.
```

Escenario 2: Límite de pedidos excedido (sp_registrar_pedido)

Se intentó abrir una sexta orden para un cliente que ya contaba con 5 pedidos pendientes. La restricción se activó exitosamente:

```
Error Code: 45000. Error: El cliente ya alcanzó el límite de 5 pedidos
pendientes.
```

Escenario 3: Actualización de inventario (sp_actualizar_stock)

Al simular la compra de 5 unidades de un producto, el sistema ejecutó el descuento exacto sobre la tabla *Productos*, validando la integridad del inventario.

Escenario 4: Actualización de estados de envío (sp_cambiar_estado_pedido)

El sistema permitió modificar de manera controlada el estado de seguimiento del pedido ID 5, pasando de 'pendiente' a 'enviado'.

Escenario 5: Borrado seguro de reseñas (sp_eliminar_reseña)

Al eliminar una calificación, el procedimiento recalculó automáticamente y devolvió el nuevo promedio general del artículo afectado.

Escenario 6: Prevención de duplicados en catálogo (sp_agregar_producto)

Se intentó insertar un producto con un nombre ya existente en la misma categoría. El candado funcionó y detuvo la operación:

```
Error Code: 45000. Error: Ya existe un producto con el mismo nombre en esta
categoría.
```

Escenario 7: Actualización de datos de contacto (sp_actualizar_cliente)

El sistema permitió la modificación del teléfono y domicilio de un cliente sin corromper el historial de pedidos asociados a su identificador.

Escenario 8: Generación de alertas de inventario (sp_reporte_stock_bajo)

La ejecución devolvió un reporte inmediato de los artículos con menos de 5 unidades, adjuntando la alerta preventiva *"¡Cuidado: Stock bajo, requiere reabastecimiento!"*.

4. Propuestas de Mejoras

Con base en el análisis de rendimiento actual, se proponen las siguientes optimizaciones para la escalabilidad del sistema:

1. **Índice Compuesto en Detalles de Pedido:** Crear un índice sobre (*id_pedido*, *id_producto*) para acelerar la generación de facturas y los cruces de información al momento de validar las reseñas.
2. **Particionamiento de Tablas (*Table Partitioning*):** Implementar particiones por rango en la tabla *Pedidos* utilizando la columna *fecha_pedido*. Esto evitará la degradación del rendimiento cuando el volumen de órdenes históricas crezca exponencialmente con los años.